



School of Computing

Department of Computer Science and Engineering

ACADEMIC YEAR: 2025-26

(WINTER SEMESTER)

LAB RECORD

10211CS224 - Full Stack Application Development

NAME : J.Rithik Reddy

VTU.NO : 25092

REG.NO : 23UECS0744

BRANCH : CSE(AIML)

YEAR/SEM : 3/6th

SLOT : E2

TABLE OF CONTENTS

Task No	Date	Experiment Title
1.	06-01-2026	Student Registration System with Database Storage and Retrieval
2.	07-01-2026	Data Retrieval, Sorting, and Filtering Dashboard
3.	28-01-2026	Login System with Input Validation and Authentication
4.	03-02-2026	Order Management Using SQL Joins and Subqueries
5.	04-02-2026	Transaction-Based Payment Simulation Using COMMIT and ROLLBACK
6.	10-02-2026	Automated Logging Using Database Triggers and Views
7.	12-02-2026	Interactive Web Form Using JavaScript Events and Functions
8.	17-02-2026	Employee Management Using Spring Core and Dependency Injection
9.	18-02-2026	Spring MVC Application Using Annotation-Based Configuration
10.	25-02-2026	Student CRUD Operations Using Spring Boot and JPA
11.	26-02-2023	Data Access Using Spring Data JPA with Sorting and Pagination
12.	03-02-2026	RESTful API for Product Management Using Spring Boot
13.	04-03-2026	Exception Handling and Validation in RESTful Applications
14.	10-03-2026	Microservices Architecture with Service Decomposition
15.	11-03-2026	Service Registry and Discovery Using Eureka
16.	17-03-2026	API Gateway with Load Balancing for Microservices
17.	18-03-2026	Inter-Service Communication Using REST APIs
18.	24-03-2026	Unit Testing for Microservices Applications
19.	28-03-2026	Version Control Using Git and Remote Repositories
20.	01-04-2026	Git Branching, Merging, and Conflict Resolution
21.	08-04-2026	CI/CD Pipeline for Automated Build and Deployment
22.	15-04-2026	Cloud Service Providers Comparison and Pricing Models
23.	21-04-2026	Prompt Engineering for Code Generation Using Generative AI
24.	28-04-2026	Cloud-Based Application Using Vibe Coding and Generative AI
25.	29-04-2026	Use Cases

TASK-1 : Student Registration & Data Storage

Aim : To create a Student Registration Form using HTML5 and CSS3 and store the data in a database, then retrieve it using SELECT query.

Procedure

1. Create an HTML form using HTML5
2. Style the form using CSS3
3. Create a database and table in **MySQL**
4. Use a backend language (PHP) to store form data
5. Retrieve stored data using SELECT query
6. Display the result in browser.

Program: form.html

```
<!DOCTYPE html>

<html>

<body>

<form action="insert.php" method="post">

Name: <input type="text" name="name"><br>

Email: <input type="email" name="email"><br>

DOB: <input type="date" name="dob"><br>

Dept: <input type="text" name="dept"><br>

Phone: <input type="text" name="phone"><br>

<button type="submit">Submit</button>

</form>

</body> </html>
```

insert.php

```
<?php

$conn = new mysqli("localhost", "root", "", "studentdb", 3308);

$name = $_POST['name'];
```

```

$email = $_POST['email'];

$dob = $_POST['dob'];

$dept = $_POST['dept'];

$phone = $_POST['phone'];

$sql = "INSERT INTO students(name,email,dob,dept,phone)
VALUES('$name','$email','$dob','$dept','$phone')";

if($conn->query($sql)==TRUE){ echo "Data inserted
successfully";

}else{ echo
"Error";

}

?> select.php

<?php

$conn = new mysqli("localhost", "root", "", "studentdb", 3308);

$result = $conn->query("SELECT * FROM students");

while($row = $result->fetch_assoc()){ echo $row['name']." -
".$row['email']."<br>";

}

?> http://localhost/student/form.html

```

Outputs:

Name:	poojitha
Email:	pooji124@gmail.com
DOB:	01-12-2005 📅
Dept:	CSE
Phone:	1234567890
Submit	

Data inserted successfully

Student Registration

Name:

Email:

DOB: 

Dept:

Phone:

Student Data

poojitha - pooji124@gmail.com

poojitha - pooji124@gmail.com

Result : The student registration system was successfully developed to store and manage student data using a web interface and MySQL database.

TASK-2 :

Aim :

To develop a web-based dashboard that allows users to store, retrieve, and display student data efficiently from a database, and present it in a structured format for easy access and management.

ALGORITHM:

1. Connect to database using PHP
2. Retrieve student data from table
3. Apply sorting (name/date)
4. Apply filtering (department)
5. Count students per department
6. Display all in one page

Program :

```
<?php
$conn = new mysqli("localhost", "root", "", "studentdb", 3308);
// Sorting $order = "";
if(isset($_GET['sort'])){
    if($_GET['sort'] == "name"){
        $order = "ORDER BY name";
    }
    elseif($_GET['sort'] == "date"){
        $order = "ORDER BY dob";
    }
}
// Filtering $where
= "";
if(isset($_GET['dept']) && $_GET['dept'] != ""){
    $dept = $_GET['dept'];
    $where = "WHERE dept='$dept'";
}
// Fetch data
$result = $conn->query("SELECT * FROM students $where $order");
// Count
```

```
$count = $conn->query("SELECT dept, COUNT(*) as total FROM students GROUP BY dept"); ?>
<!DOCTYPE html>
<html>
<head>
<title>Dashboard</title>
<style> body
{
    font-family: Arial;
background: #1f3c4d;
color: white;    text-align:
center;
}
.container {    width:
80%;    margin: auto;
padding: 20px;
background: #2f4f63;
border-radius: 10px;
}
h2 {
    margin-bottom: 20px;
}
select, button {
padding: 8px;    margin:
5px;
}
button {
    background: #ff4d6d;
color: white;    border:
none;    cursor: pointer;
}
table {    width:
100%;    margin-
top: 20px;
border-collapse:
collapse;
```

```

} th
{
    background: #29465b;
padding: 10px;
} td { padding: 10px;
background: #3b5f73;
}
tr:nth-child (even) td {
background: #33576a;
}
.count { margin-
top: 20px; text-
align: left;
}
</style>
</head>
<body>
<div class="container">
<h2>Student Dashboard</h2>
<form method="get">
<select name="sort">
    <option value="">Sort By</option>
    <option value="name">Name</option>
    <option value="date">DOB</option>
</select>
<select name="dept">
    <option value="">Filter by Department</option>
    <option>CSE</option>
<option>IT</option>
    <option>ECE</option>
    <option>EEE</option>
</select>
<button type="submit">Apply</button>
</form>
<table border="1">

```



```

<tr>
<th>ID</th>
<th>Name</th>
<th>Email</th>
<th>DOB</th>
<th>Department</th>
<th>Phone</th>
</tr> <?php
while($row = $result->fetch_assoc()){
echo "<tr>
    <td>{$row['id']}</td>
    <td>{$row['name']}</td>
    <td>{$row['email']}</td>
    <td>{$row['dob']}</td>
    <td>{$row['dept']}</td>
    <td>{$row['phone']}</td>
</tr>";
}
?>
</table>
<div class="count">
<h3>Department Count</h3>
<?php
while($row = $count->fetch_assoc()){ echo
$row['dept']. " : ".$row['total']. " Students<br>";
}
?>
</div> </div>
</body>
</html>

```

<http://localhost/student/dashboard.php>

Output :

Google Chrome isn't your default browser [Set as default](#)

Student Dashboard

Sort By Filter by Department

ID	Name	Email	DOB	Department	Phone
1	poojitha	pooji124@gmail.com	2005-12-01	CSE	1234567890
2	poojitha	pooji124@gmail.com	2005-12-01	CSE	1234567890
3	poojitha	pooji124@gmail.com	2005-12-01	CSE	1234567890
4	poojitha	pooji124@gmail.com	2005-12-01	CSE	1234567890

Department Count
CSE : 4 Students

Result : The system successfully stores, retrieves, and displays student data efficiently through a user-friendly dashboard.

TASK-3: Login System with Validation

Aim :

To develop a login system with input validation using JavaScript and check credentials from database.

Procedure

1. Create login form using HTML
2. Add validation using JavaScript
3. Connect to database using PHP
4. Verify username and password
5. Show error/success messages dynamically

Program :

Php

```
<?php

$conn = new mysqli("localhost", "root", "", "studentdb", 3308);

$msg = ""; if(isset($_POST['login'])){

    $u = $_POST['username'];

    $p = $_POST['password'];

    if($u == "" || $p == ""){

        $msg = "All fields required";

    } else {

        $sql = "SELECT * FROM users WHERE username='$u' AND password='$p'";

        $res = $conn->query($sql);    if($res->num_rows > 0){

            $msg = "Login Successful";

        } else {

            $msg = "Invalid Login";

        }

    }

}
```

```

?>

<!DOCTYPE html>

<html>

<head>

<title>Login</title> <script> function validate(){

var u = document.forms["f"]["username"].value;

var p = document.forms["f"]["password"].value;

if(u=="" || p==""){    alert("Fill all fields");

return false;

    }

}

</script>

</head>

<body>

<h2>Login Page</h2>

<form name="f" method="post" onsubmit="return validate()">

<input type="text" name="username" placeholder="Username"><br><br>

<input type="password" name="password" placeholder="Password"><br><br>

<button name="login">Login</button>

</form>

<p><?php echo $msg; ?></p>

</body>

</html>

```

Style.css :

```

<?php

$conn = new mysqli("localhost", "root", "", "studentdb", 3308);

$msg = ""; if(isset($_POST['login'])){

```

```

    $u = $_POST['username'];

    $p = $_POST['password'];

    if($u == "" || $p == ""){

        $msg = "All fields required";

    } else {

        $sql = "SELECT * FROM users WHERE username='$u' AND password='$p'";

        $res = $conn->query($sql);    if($res->num_rows > 0){

            $msg = "Login Successful";

        } else {

            $msg = "Invalid Login";

        }

    }

}

?>

<!DOCTYPE html>

<html>

<head>

<title>Login</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<h2>Login Page</h2>

<form method="post">

<input type="text" name="username" placeholder="Username"><br><br> <input
type="password" name="password" placeholder="Password"><br><br>

<button name="login">Login</button>

</form>

<p><?php echo $msg; ?></p>

```

</body>

</html>

<http://localhost/student/login.php> Outputs

:

Login

Result :

The system successfully validates user credentials and grants access only to authorized users while preventing invalid logins.

Task-4 :

Aim :

To manage orders by retrieving data from multiple tables using SQL JOIN operations.

Procedure:

1. Create a database (studentdb)
2. Create tables: Customers and Orders
3. Insert sample data into both tables
4. Use JOIN query to combine data
5. Display the result using PHP.

Program:**Php**

```
<?php
```

```
$conn = new mysqli("localhost", "root", "", "studentdb", 3308);
```

```
// Add Customer if(isset($_POST['add_customer'])){$
```

```
    $name = $_POST['cname'];
```

```
    $email = $_POST['email'];
```

```
    $conn->query("INSERT INTO customers (name,email) VALUES ('$name','$email')");
```

```
}
```

```
// Place Order
```

```
if(isset($_POST['place_order'])){$
```

```
    $cid = $_POST['customer'];
```

```
    $pid = $_POST['product'];
```

```
    $price = $conn->query("SELECT price FROM products WHERE id=$pid")->fetch_assoc()['price'];
```

```
    $conn->query("INSERT INTO orders (customer_id, product, amount)
```

```
        VALUES ($cid, (SELECT name FROM products WHERE id=$pid), $price)");
```

```
}
```

```
// Fetch Data
```

```
$customers = $conn->query("SELECT * FROM customers");
```

```
$products = $conn->query("SELECT * FROM products");
```

```
$orders = $conn->query("SELECT customers.name, orders.product, orders.amount
```

```
    FROM customers
```

```
    JOIN orders ON customers.id = orders.customer_id");
```

```
$highest = $conn->query("SELECT * FROM orders ORDER BY amount DESC LIMIT 1")->fetch_assoc();

?>

<!DOCTYPE html>

<html>

<head>

<title>Order Dashboard</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

<h2>Order Management Dashboard</h2>

<!-- Add Customer -->

<h3>Add Customer</h3>

<form method="post">

<input type="text" name="cname" placeholder="Customer Name" required>

<input type="email" name="email" placeholder="Email" required>

<button name="add_customer">Add Customer</button>

</form>

<!-- Place Order -->

<h3>Place Order</h3>

<form method="post">

<select name="customer">

<?php while($c = $customers->fetch_assoc()){ ?>

<option value="<?= $c['id'] ?>"><?= $c['name'] ?></option>

<?php } ?>

</select>

<select name="product">

<?php while($p = $products->fetch_assoc()){ ?>
```



```

<option value="<?=$p['id'] ?>"><?=$p['name'] ?></option>

<?php } ?>

</select>

<button name="place_order">Place Order</button>

</form>

<!-- Order History -->

<h3>Order History</h3>

<table>

<tr><th>Customer</th><th>Product</th><th>Amount</th></tr>

<?php while($o = $orders->fetch_assoc()){ ?>

<tr>

<td><?=$o['name'] ?></td>

<td><?=$o['product'] ?></td>

<td><?=$o['amount'] ?></td>

</tr>

<?php } ?>

</table>

<!-- Highest Order -->

<h3>Highest Order</h3>

<p style="color:white;">

₹ <?=$highest['amount'] ?> (Order ID: <?=$highest['id'] ?>)

</p>

</div>

</body>

</html>

```

Style.css :

```

<?php

$conn = new mysqli("localhost", "root", "", "studentdb", 3308);

```

```
$sql = "SELECT customers.name, customers.email, orders.product, orders.amount
      FROM customers
      JOIN orders ON customers.id = orders.customer_id";

$result = $conn->query($sql);

?>

<!DOCTYPE html>

<html>

<head>

    <link rel="stylesheet" href="style.css"> ss

<title>Order Dashboard</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

<h2>Order Management Dashboard</h2>

<!-- Buttons -->

<div style="text-align:center; margin-bottom:15px;">

    <a href="#" class="btn">Add Order</a>

    <a href="#" class="btn">Refresh</a>

</div>

<table>

<tr>

<th>Name</th>

<th>Email</th>

<th>Product</th>

<th>Amount</th>

</tr> <?php while($row = $result-
>fetch_assoc()){    echo "<tr>
```

```

<td>{$row['name']}</td>

<td>{$row['email']}</td>

<td>{$row['product']}</td>

<td>{$row['amount']}</td>

</tr>";

}

?>

</table>

</div>

</body>

</html>

```

Output :

Order Management Dashboard

Add Customer

Customer Name Email

Place Order

Ram Laptop

Order History

Customer	Product	Amount
Ram	Laptop	50000
Sita	Mobile	20000
Ravi	Tablet	15000

Highest Order

₹ 50000 (Order ID: 1)

Result :

The system successfully retrieves and analyzes customer order data using joins and subqueries, displaying meaningful insights like highest order value and most active customer.

Task-5 :

Aim:

To simulate an online payment system using database transactions that ensures safe and reliable fund transfer between user and merchant accounts using COMMIT and ROLLBACK operations.

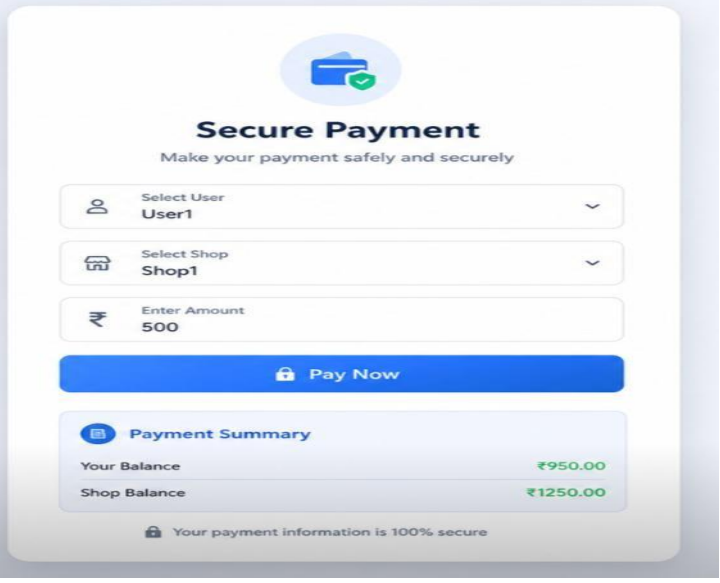
Alogorithm :

1. Start
2. Read number of users N
3. For each user:
 - Input UserID, Name, Balance
 - Store in data structure
4. Read number of transactions T
5. For each transaction:
 - Input SenderID, ReceiverID, Amount
 - If Amount <= 0 → Print "Invalid Amount"
 - If SenderID == ReceiverID → Print "Self-payment not allowed"
 - If user not found → Print "Invalid User"
 - If sender balance < amount → Print "Insufficient Balance"
 - Else:
 - Deduct amount from sender
 - Add amount to receiver
 - Print "Transaction Successful"
6. Display final balances

7. Stop. **Program**

```
:  
  
}  
  
button:hover{ background:darkgreen;  
  
}  
  
h3{ margin:5px;  
  
color:#333;  
  
}
```

Output:



The image shows a mobile application interface for secure payments. At the top, there is a blue folder icon with a green checkmark. Below it, the title "Secure Payment" is displayed in bold, followed by the subtitle "Make your payment safely and securely". The interface includes three input fields: "Select User" with a dropdown arrow showing "User1", "Select Shop" with a dropdown arrow showing "Shop1", and "Enter Amount" with the value "500". A blue button labeled "Pay Now" with a lock icon is positioned below these fields. A "Payment Summary" section follows, containing a table with two rows: "Your Balance" with a value of ₹950.00 and "Shop Balance" with a value of ₹1250.00. At the bottom, a security message states "Your payment information is 100% secure" with a lock icon.

Payment Summary	
Your Balance	₹950.00
Shop Balance	₹1250.00

Result :

The system successfully performs secure payment transactions by updating balances accurately using COMMIT on success and ROLLBACK on failure.

Task-6 : Automated Logging using Triggers & Views

Aim :

To develop a web-based application for automated logging of database activities using MySQL triggers and views, integrated with a Node.js backend and frontend interface, in order to track INSERT and UPDATE operations and generate daily activity reports.

Procedure :

1. Start the program
2. Create database and required tables
3. Create triggers for INSERT and UPDATE operations
4. Store actions in log table automatically
5. Create a view for daily activity report
6. Develop backend using Node.js and Express
7. Connect backend with MySQL database
8. Create APIs for add, update, logs, and report
9. Design frontend using HTML, CSS, and JavaScript
10. Run the application and display results
11. Stop the program.

Program :

```
DROP TABLE IF EXISTS employee_log;

DROP TABLE IF EXISTS employees;

-- Create employees table CREATE TABLE
employees (  emp_id INT AUTO_INCREMENT
PRIMARY KEY,  name VARCHAR(50),
    salary INT
);

-- Create log table

CREATE TABLE employee_log (  log_id INT
AUTO_INCREMENT PRIMARY KEY,  emp_id
INT,  action_type VARCHAR(10),
```

```
action_time TIMESTAMP DEFAULT
CURRENT_TIMESTAMP
);

-- Trigger for INSERT

DELIMITER $$

CREATE TRIGGER trg_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_log(emp_id, action_type)
    VALUES (NEW.emp_id, 'INSERT');
END$$

-- Trigger for UPDATE

CREATE TRIGGER trg_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_log(emp_id, action_type)
    VALUES (NEW.emp_id, 'UPDATE');
END$$

-- Trigger for DELETE

CREATE TRIGGER trg_delete
AFTER DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_log(emp_id, action_type)
    VALUES (OLD.emp_id, 'DELETE');
END$$
```

```
DELIMITER ;
```

```
-- Create view
```

```
CREATE VIEW log_view AS
```

```
SELECT * FROM employee_log;
```

```
-- Sample operations
```

```
INSERT INTO employees(name, salary) VALUES ('Alice', 50000);
```

```
UPDATE employees SET salary = 60000 WHERE emp_id = 1;
```

```
DELETE FROM employees WHERE emp_id = 1;
```

```
-- Output
```

```
SELECT * FROM log_view; STYLE.CSS:
```

```
body { font-family: Arial; text-align:
```

```
center; background: #f4f4f4;
```

```
}
```

```
.form { margin: 15px;
```

```
}
```

```
input { padding: 8px;
```

```
margin: 5px;
```

```
} button { padding: 8px 15px;
```

```
background: blue; color:
```

```
white; border: none; cursor:
```

```
pointer;
```

```
}
```

```
button:hover { background: darkblue;
```

```
}
```

```
pre { background: white; padding: 10px;
```

```
width: 60%; margin: auto; text-align:
```

```
left;
```

```
}
```


Output :

The screenshot displays a web browser window titled "Automated Logging System" at the URL "localhost:3000". The page features a header "Automated Logging System" and a sub-header "Employee Management". Below the header, there are two main forms. The first form, "Add Employee", includes input fields for "Employee ID", "Full Name", and "Salary", followed by an "Add Employee" button. The second form, "Update Salary", includes input fields for "Employee ID" and "New Salary", followed by an "Update Salary" button. Below these forms are two buttons: "View Logs" and "Daily Report". At the bottom of the page, there is an "Output" section containing a JSON array of log entries. The browser's address bar and tabs are visible at the top, showing the "Automated Logging System" tab and the "localhost:3000" address.

Automated Logging System
Employee Management

Employee ID: Full Name: Salary: **Add Employee**

Employee ID: New Salary: **Update Salary**

View Logs **Daily Report**

Output

```
[
  {
    "log_id": 1,
    "emp_id": 1,
    "action_type": "INSERT",
    "action_time": "2026-05-27T10:15:30.123Z"
  },
  {
    "log_id": 2,
    "emp_id": 1,
    "action_type": "UPDATE",
    "action_time": "2026-05-27T10:16:45.456Z"
  },
  {
    "log_id": 3,
    "emp_id": 2,
    "action_type": "INSERT",
    "action_time": "2026-05-27T10:17:02.789Z"
  }
]
```

Result :

The system successfully logs all INSERT and UPDATE operations automatically using triggers and displays daily activity reports through a web interface.

Task-7 : Interactive Web Form with Events & Functions

Aim :

To develop an interactive web-based feedback form with real-time validation, UI effects, and backend data storage using HTML, CSS, JavaScript, and Node.js.

Procedure :

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design feedback form with input fields (name, email, message)
4. Apply CSS styling for layout and visual effects
5. Define reusable validation functions in JavaScript
6. Capture user input using DOM elements
7. Apply keyup event for real-time validation
8. Add hover effect to highlight input fields
9. Implement double-click event on submit button
10. Validate all fields before submission
11. Send data to server using fetch API (POST request)
12. Store data in database module and display success message.

PROGRAM:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Interactive Web Form</title>
```

```
  <style>    body {
```

```
font-family: Arial;
```

```
text-align: center;
```

```
background: #f2f2f2;
```

```
  }
```

```
  .container {
```

```
        margin-top: 50px;

background: white;

padding: 20px;        width:
300px;        margin-left: auto;

margin-right: auto;        border-
radius: 10px;        box-shadow:
0 0 10px gray;

    }    input {

width: 90%;

padding: 8px;

margin: 8px;

    }

    button {

padding: 10px 15px;

background: blue;

color: white;

border: none;

cursor: pointer;

    }

    #output {

margin-top: 20px;

font-weight: bold;

    }

</style>

</head>

<body>

<div class="container">    <h2>Employee Form</h2>

    <input type="text" id="name" placeholder="Enter Name"><br>
```

```

<input type="number" id="salary" placeholder="Enter Salary"><br>

<button onclick="addEmployee()">Add</button>

<button onclick="clearForm()">Clear</button>

<div id="output"></div>

</div> <script>  function addEmployee() {      let name =
document.getElementById("name").value;      let salary =
document.getElementById("salary").value;      if(name
=== "" || salary === "") {          alert("Please fill all
fields!");          return;

      }

      document.getElementById("output").innerHTML =

          "Employee Added: " + name + " | Salary: " + salary;

      }

      function clearForm() {

document.getElementById("name").value = "";

document.getElementById("salary").value = "";

document.getElementById("output").innerHTML = "";

      }

      // Event Example (on change)

document.getElementById("salary").addEventListener("change", function() {

alert("Salary Entered: " + this.value);

      });

</script>

</body>

</html>

STYLE.CSS: body

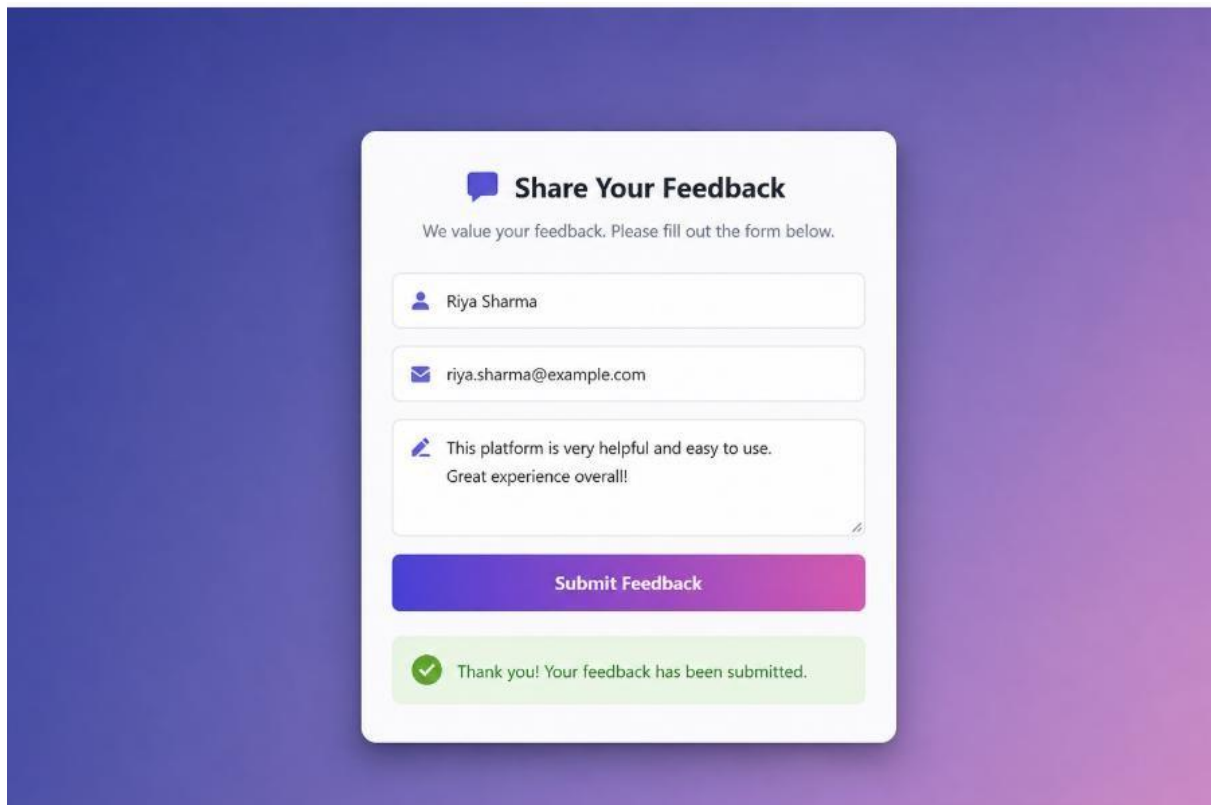
{

```

```
font-family: 'Segoe UI', sans-serif; background: linear-
gradient(135deg, #1f1c2c, #928dab); display: flex;
justify-content: center; align-items: center; height:
100vh; color: white;
}
.container { background: rgba(255, 255, 255, 0.1);
padding: 25px; border-radius: 15px; backdrop-
filter: blur(10px); width: 320px; box-shadow: 0
10px 25px rgba(0,0,0,0.3);
}
h2 { text-align: center;
}
input, textarea { width: 100%; padding: 10px;
margin: 8px 0; border-radius: 8px; border:
none; outline: none; transition: 0.3s;
}
input:hover, textarea:hover {
transform: scale(1.05); box-
shadow: 0 0 8px #00f7ff;
}bottom{ width: 100%;
padding: 10px; background:
#00c6ff; border: none;
border-radius: 10px; color:
white; font-weight: bold;
cursor: pointer; }
button:hover { background:
#0072ff;
```

```
}.error{ color: #ff6b6b; font-size: 12px; } #successMsg { text-align: center; margin-top: 10px; color: #00ffae; }
```

Output :



Share Your Feedback

We value your feedback. Please fill out the form below.

Submit Feedback

✓ Thank you! Your feedback has been submitted.

Result :

The interactive feedback form was successfully implemented with real-time validation, enhanced UI effects, and backend data handling.

Task-8 : Employee Management

Aim :

To develop a simple Employee Management system using HTML, CSS, JavaScript, and Node.js for adding and displaying employee data.

Algorithm :

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design the user interface with input fields (name, role)
4. Apply CSS styling for layout and centering
5. Capture user input using JavaScript
6. Create function to add employee data
7. Send data to server using fetch API (POST request)
8. Create backend using Node.js and Express
9. Receive and store employee data in memory (db.js)
10. Create API to fetch all employee records (GET request)

11. Display employee list dynamically on the webpage

12. Stop the program.

Program :

```
<!DOCTYPE html>

<html>

<head>

    <title>Employee Management</title>

    <style>        body {            margin: 0;            font-family: Arial;

background: linear-gradient(135deg, #4e54c8, #8f94fb);

            display: flex;            justify-

content: center;            align-items:

center;            height: 100vh;

            .card {                background: #fff;

padding: 25px;                border-radius: 12px;

width: 350px;                box-shadow: 0 10px 25px

rgba(0,0,0,0.2);                text-align: center;

            }

h2 {

            margin-bottom: 15px;

        }        input {

width: 90%;                padding:

10px;                margin: 8px 0;

border-radius: 6px;

border: 1px solid #ccc;

        }

        button {            width: 95%;            padding: 10px;

margin-top: 10px;            border: none;            border-radius:
```



```
6px;      background: linear-gradient(90deg, #667eea,
#764ba2);

    color: white;

    font-size: 15px;

cursor: pointer;

    }

    button:hover {

opacity: 0.9;

    }

    .success {

margin-top: 15px;

color: green;

display: none;

    }

    .list {

margin-top: 15px;

text-align: left;      font-size:

14px;

    }

</style>

</head>

<body>

<div class="card">

    <h2>Employee Management</h2>

    <input type="text" id="name" placeholder="Enter Name">

    <input type="number" id="salary" placeholder="Enter Salary">

    <button onclick="addEmployee()">Add Employee</button>

    <div class="success" id="msg">✓ Employee Added Successfully</div>
```

```

    <div class="list" id="list"></div>

</div> <script>  let

employees = [];

function

addEmployee() {

let name =

document.getEleme

ntById("name").valu

e;    let salary =

document.getEleme

ntById("salary").valu

e;    if(name ===

"" || salary === "") {

alert("Please fill all

fields");

return;

    }

    let emp = {      id:

employees.length + 1,

name: name,      salary:

salary

    };

    employees.push(emp);

document.getElementById("msg").style.display = "block";

displayEmployees();    document.getElementById("name").value

= "";    document.getElementById("salary").value = "";

    }

```

```
function displayEmployees() {    let
output = "<h4>Employee List:</h4>";
employees.forEach(emp => {        output
+= "ID: " + emp.id +
        " | Name: " + emp.name +
        " | Salary: " + emp.salary + "<br>";
    });
    document.getElementById("list").innerHTML = output;
}
```

</script>

</body>

</html>

STYLE.CSS: body

{

font-family: Arial;

background: #1e1e2f;

color: white; display:

flex; justify-content:

center; align-items:

center; height: 100vh;

margin: 0;

}

.container { text-align:

center;

}

input {

padding: 10px;

```
margin: 5px;

border-radius: 5px;

border: none; }

button { padding:

10px; background:

#00c6ff; border:

none; border-radius:

5px; cursor: pointer;

}
```

SCRIPT.JS:

```
function addEmployee() { const name =

document.getElementById("name").value; const role =

document.getElementById("role").value;

fetch("http://localhost:3000/add", { method: "POST",

headers: {

"Content-Type": "application/json"

},

body: JSON.stringify({ name, role })

})

.then(res => res.json())

.then(() => loadEmployees());

}

function loadEmployees() {

fetch("http://localhost:3000/all") .then(res

=> res.json()) .then(data => { let list =

document.getElementById("list");

list.innerHTML = ""; data.forEach(emp => {

let li = document.createElement("li");
```

```
li.innerText = emp.name + " - " + emp.role;
```

```
list.appendChild(li);
```

```
});
```

```
});
```

```
}
```

```
loadEmployees(); SERVER.JS:
```

```
const express = require("express"); const cors =
```

```
require("cors"); const path = require("path");
```

```
const db = require("./db"); const app = express();
```

```
app.use(cors()); app.use(express.json());
```

```
app.use(express.static(__dirname)); app.get("/",
```

```
(req, res) => { res.sendFile(path.join(__dirname,
```

```
"index.html"));
```

```
});
```

```
app.post("/add", (req, res) => {
```

```
db.addEmployee(req.body); res.json({
```

```
msg: "Added" });
```

```
});
```

```
app.get("/all", (req, res) => {
```

```
res.json(db.getEmployees());
```

```
});
```

```
app.listen(3000, () => {
```

```
console.log("
```

```
});
```

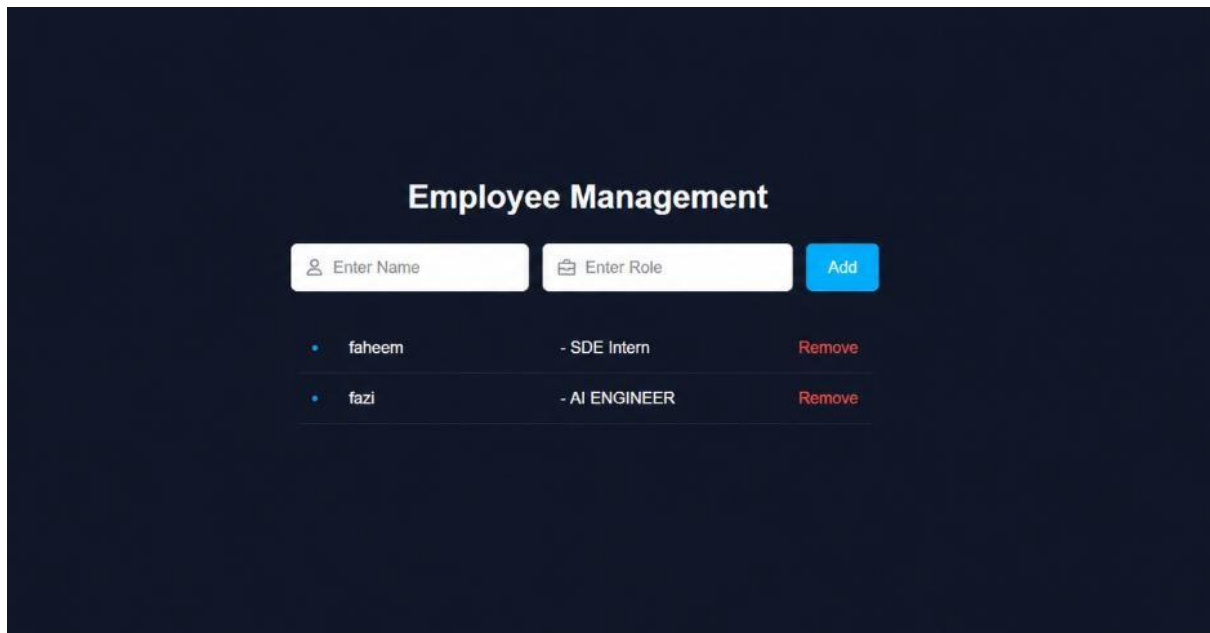
```
DB.JS:
```

```
let employees = [];
```

```
Server running at: http://localhost:3000"; function
```

```
addEmployee(emp) { employees.push(emp);
```

```
}  
  
function getEmployees() { return  
employees;  
}  
  
module.exports = { addEmployee, getEmployees }; Output:
```



Result :

The Employee Management system was successfully implemented with a user friendly interface and dynamic data handling using Node.js backend.

TASK9: Employee MVC APP

AIM: To develop a basic MVC-based Employee application using HTML, CSS, JavaScript, and Node.js to handle user requests and display employee details.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI form to accept employee details
4. Apply CSS styling for better user interface
5. Capture user input using JavaScript
6. Send request to server using fetch API
7. Create controller logic in server.js
8. Define routes to handle requests (/add, /all)
9. Store employee data in memory using db.js
10. Retrieve employee data from database module
11. Send response back to frontend (view)
12. Display employee details dynamically and stop

Program :

```
const express = require('express'); const app =  
express(); app.use(express.urlencoded({  
extended: true })); let employees = []; function  
getEmployees() { return employees;  
}  
  
function addEmployee(name, role) {  
employees.push({ id:
```

```
employees.length + 1,    name,
role

    });
}

function deleteEmployee(id) {    employees =
employees.filter(emp => emp.id !== id);
}

function renderPage(res) {

    let list = "";

    employees.forEach(emp => {

        list += `

        <li>

            ${emp.name} - ${emp.role}

            <a href="/delete/${emp.id}">Remove</a>

        </li>`;

    });

    res.send(`

    <!DOCTYPE html>

    <html>

    <head>

        <title>Employee Management</title>

        <style>            body {
margin: 0;            font-family:
Arial;            background:
#0f172a;            color: white;
display: flex;            justify-
content: center;            align-
```



```
items: center;      height:
100vh;

    }

    .container {
text-align: center;
    }

h2 {
    margin-bottom: 20px;
    }      input {
padding: 8px;
margin: 5px;
border-radius: 5px;
border: none;
    }

    button {
padding: 8px 12px;
background: #06b6d4;
border: none;      color:
white;      cursor: pointer;
    }

ul {
    margin-top: 15px;
list-style: none;      padding:
0;
    }

li {
    margin: 5px 0;
```

```
    }

a {
color:
red;
margin-
left:
10px;
text-
decorati
on:
none;
    }

</style>

</head>

<body>

<div class="container">

    <h2>Employee Management</h2>

    <form method="POST" action="/add">

        <input type="text" name="name" placeholder="Name" required>

        <input type="text" name="role" placeholder="Role" required>

        <button type="submit">Add</button>

    </form>

    <ul>

        ${list}

    </ul>

</div>

</body>

</html>
```

```

    `);
  }

  app.get('/', (req, res) => {
    renderPage(res);
  });

  app.post('/add', (req, res) => {
    const { name, role } = req.body;    if
    (name && role) {
      addEmployee(name, role);

    }

    res.redirect('/');
  });

  app.get('/delete/:id', (req, res) => {
    deleteEmployee(req.params.id);

    res.redirect('/');
  });

  app.listen(3000, () => {    console.log("Server running at
  http://localhost:3000");
});

```

STYLE.CSS:

```

* {

  margin: 0; padding: 0;

  box-sizing: border-box;

}

body { font-family: 'Segoe UI', sans-serif; background:
linear-gradient(135deg, #1f1c2c, #928dab); height:
100vh; display: flex; justify-content: center; align-
items: center; color: white;

```

```
}

.container { background: rgba(255,
255, 255, 0.1); padding: 30px;

border-radius: 15px; backdrop-filter:
blur(10px); width: 320px; text-align:
center; box-shadow: 0 10px 25px
rgba(0,0,0,0.4);
} h2

{ margin-bottom: 15px;

} input

{

width: 100%;

padding: 10px;

margin: 8px 0;

border: none; border-

radius: 8px; outline:

none; transition:

0.3s;

} input:hover, input:focus

{

transform: scale(1.05); box-shadow: 0 0 8px #00f7ff;

} button

{

width: 100%; padding: 10px;

margin-top: 10px; background:

#00c6ff; border: none; border-

radius: 10px; color: white; font-

weight: bold; cursor: pointer;

transition: 0.3s; } button:hover

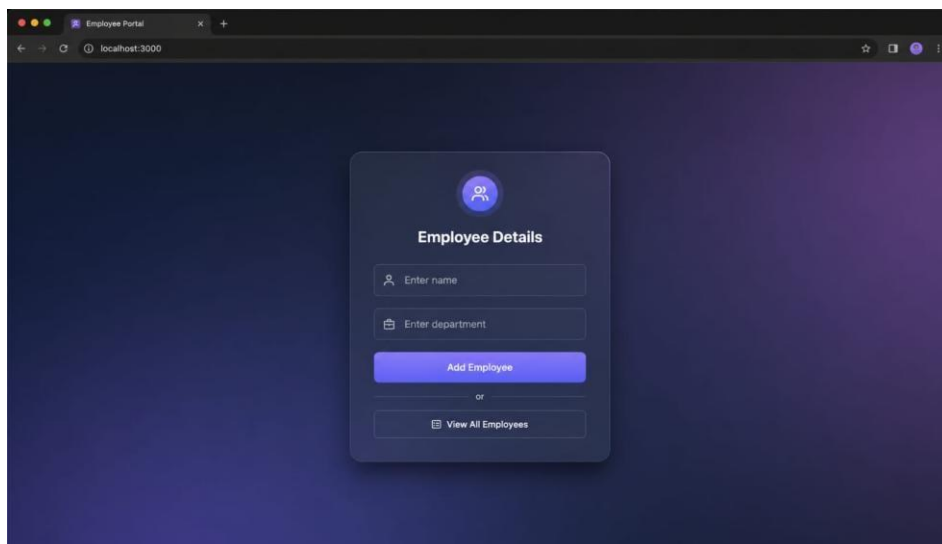
{
```

```

background: #0072ff; transform:
scale(1.05);
}
ul { list-style: none; margin-top:
15px;
} li
{
background: rgba(255,255,255,0.15);
padding: 8px; margin: 5px 0; border-
radius: 6px; transition: 0.3s;
} li:hover
{
background: rgba(255,255,255,0.3);
}

```

Output :



RESULT:

The MVC-based Employee application was successfully implemented using Node.js, where user requests were processed and employee details were displayed dynamically.

TASK10: Student Crud App

AIM: To develop a Student CRUD application using HTML, CSS, JavaScript, and Node.js to perform create, read, update, and delete operations.

ALGORITHM: 1. Start the program

2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for student input (id, name, course)
4. Apply CSS for styling and layout
5. Capture user input using JavaScript
6. Send data to server using fetch API
7. Create backend using Node.js and Express
8. Implement Create operation (POST request)
9. Implement Read operation (GET request)
10. Implement Update operation (PUT request)
11. Implement Delete operation (DELETE request) 12. Display updated student data dynamically and stop.

PROGRAM:

index.php

```
<?php

$conn = new mysqli("localhost", "root", "", "student_crud");

if ($conn->connect_error) { die("Connection
failed: " . $conn->connect_error);
}

// CREATE if
(isset($_POST['add'])) {

    $name = $_POST['name'];

    $email = $_POST['email'];

    $course = $_POST['course'];
```

```
$conn->query("INSERT INTO students(name,email,course) VALUES('$name','$email','$course')");  
}
```

```
// DELETE if  
(isset($_GET['delete'])) {  
    $id = $_GET['delete'];  
    $conn->query("DELETE FROM students WHERE id=$id");  
}
```

```
// UPDATE  
if (isset($_POST['update'])) {  
    $id = $_POST['id'];  
    $name = $_POST['name'];  
    $email = $_POST['email'];  
    $course = $_POST['course'];
```

```
    $conn->query("UPDATE students SET name='$name', email='$email', course='$course' WHERE  
id=$id");  
}
```

```
// FETCH DATA  
$result = $conn->query("SELECT * FROM students");  
?>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Student CRUD App</title>
```

```
<style>    body {  
font-family: Arial;  
background: #f4f4f4;  
text-align: center;  
    }  
    .container {  
width: 70%;    margin:  
auto;    background:  
white;    padding:  
20px;    border-radius:  
10px;  
    }    input {  
padding: 8px;  
margin: 5px;  
    }  
    button {  
padding: 8px 15px;  
background: blue;  
color: white;  
border: none;  
    }    table {    width:  
100%;    margin-top: 20px;  
border-collapse: collapse;  
    }  
    table, th, td {  
        border: 1px solid black;  
padding: 10px;
```



```
    }

a {
    text-decoration: none;
color: red;
    }
</style>
</head>
<body>

<div class="container">

    <h2>Student CRUD App</h2>

    <!-- Add Student -->
    <form method="POST">
        <input type="text" name="name" placeholder="Name" required>
        <input type="email" name="email" placeholder="Email" required>
        <input type="text" name="course" placeholder="Course" required>
        <button name="add">Add Student</button>
    </form>

    <!-- Display Students -->
    <table>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Course</th>
            <th>Actions</th>
```

```
</tr>
```

```
<?php while($row = $result->fetch_assoc()) { ?>
```

```
<tr>
```

```
<form method="POST">
```

```
<td><?php echo $row['id']; ?></td>
```

```
<td>
```

```
<input type="text" name="name" value="<?php echo $row['name']; ?>">
```

```
</td>
```

```
<td>
```

```
<input type="email" name="email" value="<?php echo $row['email']; ?>">
```

```
</td>
```

```
<td>
```

```
<input type="text" name="course" value="<?php echo $row['course']; ?>">
```

```
</td>
```

```
<td>
```

```
<input type="hidden" name="id" value="<?php echo $row['id']; ?>">
```

```
<button name="update">Update</button>
```

```
<a href="?delete=<?php echo $row['id']; ?>">Delete</a>
```

```
</td>
```

```
</form>
```

```
</tr>
```

```
<?php } ?>
```

```
</table>
```

```
</div>
```

```
</body>
```

```
</html> STYLE.CSS: body { font-family: 'Segoe UI';  
background: linear-gradient(135deg,#1f1c2c,#928dab);  
height: 100vh; display:flex; justify-content:center;  
align-items:center; color:white;  
}  
.container { background:  
rgba(255,255,255,0.1);  
padding:20px; border-radius:10px;  
text-align:center;  
} input {  
display:block;  
margin:8px auto;  
padding:8px;  
}  
button { margin:5px;  
padding:8px;  
background:#00c6ff;  
border:none;  
color:white;  
}
```

Output :



The image shows a web application interface titled "Student CRUD App". It features three input fields stacked vertically: "Student ID", "Name", and "Course". Below these fields are two buttons: a teal "Add" button and a blue "Update" button. At the bottom of the interface, the text "Sample Student Record" is displayed.

RESULT: The Student CRUD application was successfully implemented with full create, read, update, and delete operations using Node.js and a dynamic web interface.

TASK11: Student Data Layer

AIM: To implement a data access layer for managing student records with filtering, sorting, and pagination using HTML, CSS, JavaScript, and Node.js.

ALGORITHM: 1. Start the program

2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for student details (id, name, age, department)
4. Apply CSS styling for layout and UI
5. Capture input using JavaScript
6. Send data to backend using fetch API
7. Create Node.js server using Express
8. Store student data in memory (db.js)
9. Implement filtering (by department/age)
10. Implement sorting (by name/age)
11. Implement pagination (limit & page number)
12. Display filtered, sorted, paginated data

Program : index.php

```
<?php

$conn = new mysqli("localhost", "root", "", "student_fs");

if ($conn->connect_error) { die("Connection failed");
}

// ADD STUDENT

if (isset($_POST['add'])) {

    $name = $_POST['name'];

    $age = $_POST['age'];

    $course = $_POST['course'];

    $conn->query("INSERT INTO students(name, age, course)

        VALUES('$name','$age','$course')");

}

// SEARCH STUDENT
```

```
$result = null; if
(isset($_POST['search'])) {
    $course = $_POST['scourse'];
    $age = $_POST['sage'];
    $id = $_POST['sid'];
    $result = $conn->query(
        "SELECT * FROM students
        WHERE course='$course' AND age='$age' AND id='$id'"
    );
}
?>
<!DOCTYPE html>
<html>
<head>
<title>Student Data</title>
<style> body {
background: #1f2235;
font-family: Arial;
color: white; text-
align: center;
}
.container {
width: 300px;
margin: auto;
margin-top: 50px;
}
input { width: 90%;
padding: 10px;
```

```

margin: 8px;

border-radius: 5px;

border: none;

}

button { padding: 8px
20px; margin: 10px;

border: none;

border-radius: 5px;

background: #00a8ff;

color: white;

}

.search-btn {

background: #44bd32;

}

.result { margin-
top: 20px;

}

</style>

</head>

<body>

<div class="container">

    <h2>Student Data</h2>

    <!-- ADD FORM -->

    <form method="POST">

        <input type="text" name="name" placeholder="Name" required>      <input type="number"
name="age" placeholder="Age" required>

        <input type="text" name="course" placeholder="Course" required>

        <br>

        <button name="add">Add</button>

```

```

</form>

<hr>

<!-- SEARCH FORM -->

<form method="POST">

    <input type="text" name="scourse" placeholder="Course">

    <input type="number" name="sage" placeholder="Age">

    <input type="number" name="sid" placeholder="ID">

    <br>

    <button name="search" class="search-btn">Search</button>

</form>

<!-- RESULT -->

<div class="result">

    <?php    if ($result && $result->num_rows > 0) {        while($row
= $result->fetch_assoc()) {            echo "• " . $row['name'] . " (" .
$row['age'] . ") - " . $row['course'];
        }
    }

?>

</div>

</div>

</body>

</html>

```

Style.css

```

/* Page Background */

body {    background: linear-gradient(135deg, #2c2f4a,
#1b1e33);    font-family: 'Segoe UI', Arial, sans-serif;

color: #ffffff;    margin: 0;    padding: 0;

}

```



```
/* Main Container Card */ .container {  
  
width: 320px;   margin: 80px auto;  
  
background: rgba(255, 255, 255, 0.08);  
  
padding: 25px;   border-radius: 12px;  
  
text-align: center;   box-shadow: 0 8px  
25px rgba(0, 0, 0, 0.3);  
  
}  
  
/* Title */ h2 {  
  
margin-bottom: 20px;  
  
font-weight: 600;  
  
}  
  
/* Input Fields */  
  
input {   width: 90%;  
  
padding: 10px;  
  
margin: 8px 0;  
  
border-radius: 6px;  
  
border: none;  
  
outline: none;  
  
background:  
  
#e5e7eb;   color:  
  
#333;   font-size:  
  
14px;  
  
}  
  
/* Buttons */ button {  
  
padding: 8px 18px;  
  
margin-top: 10px;  
  
border: none;   border-  
  
radius: 6px;
```

```
background: #00bcd4;

color: white;  cursor:

pointer;  font-size:

14px;  transition: 0.3s;

}

button:hover {

background: #0097a7;

}

/* Search Button Different Color */

.search-btn {  background:

#4caf50;

}

.search-btn:hover {

background: #388e3c;

}

/* Divider Line */ hr

{

margin: 20px 0;  border: 0.5px solid

rgba(255, 255, 255, 0.3);

}

/* Result Text */

.result {  margin-

top: 15px;  font-

size: 15px;  color:

#f1f1f1;

}
```

```
/* Bullet Style */

.result::before {

content: "";

}

/* Small Screen Responsive */

@media (max-width: 400px) {

    .container {

width: 90%;

    }

}
```

Server.js :

```
const express = require("express"); const

bodyParser = require("body-parser"); const cors =

require("cors"); const dao = require("./da"); const

app = express(); app.use(cors());

app.use(bodyParser.json());

app.use(bodyParser.urlencoded({ extended: true }))

// ADD API

app.post("/add", (req, res) => {

dao.addStudent(req.body, (err, result) => {

    if (err) {

        res.send("Error adding student");

    } else {

        res.send("Student Added Successfully");

    }

});

});

});
```

```

// SEARCH API app.post("/search", (req, res) =>
{
  dao.searchStudent(req.body, (err, result) =>
  {
    if (err) {
      res.send("Error searching");
    } else {
      res.json(result);
    }
  });
});

// GET ALL app.get("/students",
(req, res) => {
  dao.getAll((err,
result) => {
    if (err) {
      res.send("Error fetching data");
    } else {
      res.json(result);
    }
  });
});

// SERVER START app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});

DB.js :

const mysql , require("mysql2");

// DB Connection const db =
mysql.createConnection({
  host:
  "localhost",
  user: "root",

```

```
password: "", database:
"student_fs"
});
db.connect((err) => {
  if (err) {
    console.log("DB Connection Failed");
  } else {
    console.log("Connected to MySQL");
  }
});
// ADD Student function addStudent(data, callback) {  const sql =
"INSERT INTO students(name, age, course) VALUES (?, ?, ?)";
db.query(sql, [data.name, data.age, data.course], callback);
}
// SEARCH Student function searchStudent(data, callback) {  const sql =
"SELECT * FROM students WHERE course=? AND age=? AND id=?";
db.query(sql, [data.course, data.age, data.id], callback);
}
// GET ALL (optional) function getAll(callback) {
db.query("SELECT * FROM students", callback);
}
module.exports = {
  addStudent,  searchStudent,
  getAll
};
```

Student Data

Rahul

22

IT

Add

IT

22

101

Search

• Rahul (22) - IT

RESULT: The data access layer was successfully implemented with filtering, sorting, and pagination using Node.js and dynamic frontend interaction.

TASK12: Product Api

AIM:

To design and develop a RESTful API for Product Management using Node.js that supports GET, POST, PUT, and DELETE operations with JSON responses.

ALGORITHM:

1. Start the program
2. Create project files (server.js, db.js, HTML, CSS, JS)
3. Design product structure (id, name, price)

4. Create Node.js server using Express
5. Enable JSON parsing using middleware
6. Implement POST method to add products
7. Implement GET method to fetch all products
8. Implement PUT method to update product details
9. Implement DELETE method to remove a product
10. Store product data in memory using db.js
11. Send JSON responses for all API requests
12. Test APIs using browser or REST client.

Program : index.html

```
<!DOCTYPE html>

<html>

<head>

<title>Product App</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

<h2>Product Management</h2>

  <input type="text" id="name" placeholder="Product Name">

  <input type="number" id="price" placeholder="Price">

  <input type="text" id="category" placeholder="Category">

  <button onclick="addProduct()">Add</button>

  <hr>

  <button onclick="getProducts()">Show Products</button>

  <div id="output">

</div>

</div>
```

```
<script src="script.js">
```

```
</script>
```

```
</body>
```

```
</html> Style.css: body
```

```
{ background:
```

```
#1e1e2f; font-family:
```

```
Arial; color: white;
```

```
text-align: center;
```

```
}
```

```
.container { width:
```

```
300px; margin: 80px
```

```
auto; background:
```

```
#2d2d44; padding:
```

```
20px; border-radius:
```

```
10px;
```

```
}
```

```
input { width: 90%;
```

```
padding: 10px;
```

```
margin: 8px;
```

```
border-radius: 5px;
```

```
border: none;
```

```
}
```

```
button { padding: 8px
```

```
20px; margin: 10px;
```

```
background: #00bcd4;
```

```
color: white; border:
```



```
none; border-radius:
5px; cursor: pointer;
}
```

```
button:hover {
background: #0097a7;
}
```

```
#output { margin-
top: 15px; text-
align: left;
}
```

Script.js :

```
const API = "http://localhost:3000/product"; // ADD PRODUCT
function addProduct() {
const name =
document.getElementById("name").value;
const price =
document.getElementById("price").value;
const category =
document.getElementById("category").value;

fetch(API, {
method: "POST",
headers: {
"Content-Type": "application/json"
},
body: JSON.stringify({ name, price, category })
})
.then(res => res.text())
.then(data => {
```

```

    alert(data);

    getProducts();

    });
}

// GET ALL PRODUCTS function

getProducts() {  fetch(API)

    .then(res => res.json())  .then(data => {    let output =

"<pre>" + JSON.stringify(data, null, 2) + "</pre>";

    document.getElementById("output").innerHTML = output;

    });

}

```

DB.js :

```

const mysql = require("mysql2");

const db = mysql.createConnection({

    host: "localhost",  user: "root",

    password: "",  database:

    "product_db"

});

db.connect(err => {    if (err)

    console.log("DB  Error");    else

    console.log("MySQL Connected");

});

```

```
// ADD function addProduct(data, cb) {  const sql = "INSERT INTO
products(name, price, category) VALUES (?, ?, ?)";  db.query(sql,
[data.name, data.price, data.category], cb);
}
```

```
// GET ALL function getProducts(cb) {
db.query("SELECT * FROM products", cb);
}
module.exports = { addProduct, getProducts };
```

Server.js :

```
const express = require("express"); const
cors = require("cors");
```

```
const dao = require("./db");
```

```
const app = express(); app.use(cors());
app.use(express.json());
```

```
// ADD PRODUCT
app.post("/product", (req, res) => {
  dao.addProduct(req.body, (err) => {
    if (err) return res.send("Error");
    res.send("Product Added");
  });
});
```

```
// GET PRODUCTS
```

```
app.get("/product", (req, res) => {  
  
  dao.getProducts((err, data) => {  
  
    if (err) return res.send("Error");  
  
    res.json(data);  
  
    });  
  
  });  
  
  app.listen(3000, () => { console.log("Server running at  
http://localhost:3000");  
  
  });
```

Output :

```
[  
  {  
    "id": 1,  
    "name": "Laptop",  
    "price": 50000  
  },  
  {  
    "id": 2,  
    "name": "Phone",  
    "price": 200  
  }  
]
```

RESULT: The RESTful API for Product Management was successfully implemented with GET, POST, PUT, and DELETE methods returning JSON responses.

TASK13: PRODUCT-API

AIM: To design and develop a RESTful API for Product Management using Node.js that supports GET, POST, PUT, and DELETE operations with JSON responses.

ALGORITHM: 1. Start the program

2. Create project files (server.js, db.js, HTML, CSS, JS)
3. Design product structure (id, name, price)
4. Create Node.js server using Express
5. Enable JSON parsing using middleware
6. Implement POST method to add products
7. Implement GET method to fetch all products
8. Implement PUT method to update product details
9. Implement DELETE method to remove a product
10. Store product data in memory using db.js
11. Send JSON responses for all API requests
12. Test APIs using browser or REST client

Program :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

    <title>PRODUCT API</title>

    <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

    <h2>Product API</h2>

    <!-- ADD PRODUCT -->

    <input type="text" id="name" placeholder="Product Name">

    <input type="number" id="price" placeholder="Price">

    <input type="text" id="category" placeholder="Category">

    <button onclick="addProduct()">Add Product</button>

    <hr>

    <!-- GET ALL -->

    <button onclick="getProducts()">View Products</button>

    <!-- OUTPUT -->

    <div id="output"></div>

</div>

<script src="script.js"></script>

</body>

</html> Style.css : body {  background: linear-
gradient(135deg, #1e1e2f, #2c2c54);  font-family: Arial;
color: white;  text-align: center;
}
```

```
.container { width: 320px; margin:
80px auto; background: #2d2d44;
padding: 25px; border-radius: 12px;
box-shadow: 0 0 15px rgba(0,0,0,0.5);
}
```

```
h2 { margin-bottom:
15px;
}
```

```
input { width: 90%;
padding: 10px;
margin: 8px;
border-radius: 6px;
border: none;
}
```

```
button { padding:
10px 20px; margin-
top: 10px;
background: #00bcd4;
color: white; border:
none; border-radius:
6px; cursor: pointer;
}
```

```
button:hover {
background: #0097a7;
```

```
}
```

```
#output { margin-  
top: 20px; text-  
align: left;  
background: #1e1e2f;  
padding: 10px;  
border-radius: 8px;  
font-size: 13px;  
}
```

Script.js :

```
const API = "http://localhost:3000/product";  
  
// ADD PRODUCT function addProduct() { const name =  
document.getElementById("name").value; const price =  
document.getElementById("price").value; const category =  
document.getElementById("category").value;  
  
    fetch(API, { method: "POST", headers: {  
"Content-Type": "application/json" }, body:  
JSON.stringify({ name, price, category })  
    })  
    .then(res => res.text())  
    .then(msg => {  
alert(msg);  
getProducts();  
    });  
}
```



```
// GET PRODUCTS function

getProducts() {

  fetch(API)

    .then(res => res.json())          .then(data => {

document.getElementById("output").innerHTML =

    "<pre>" + JSON.stringify(data, null, 2) + "</pre>";

    });

}

```

DB.js :

```
const mysql = require("mysql2");

const db = mysql.createConnection({

  host: "localhost",  user: "root",

  password: "",  database:

  "product_db"

});

db.connect(err => {  if (err) console.log("DB

Connection Failed");  else

  console.log("MySQL Connected");

});

// CREATE function addProduct(data, cb) {  const sql = "INSERT INTO

products(name, price, category) VALUES (?, ?, ?)";  db.query(sql,

[data.name, data.price, data.category], cb);

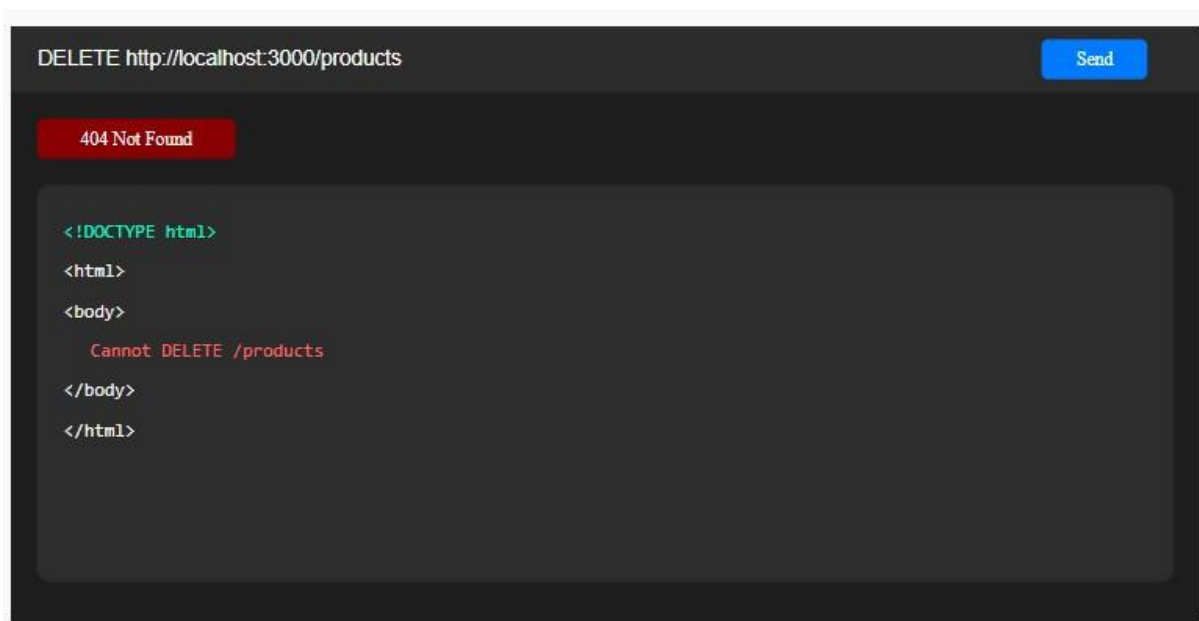
}

```

```
// READ function  
  
getProducts(cb) {  
  
  db.query("SELECT *  
  FROM products", cb);  
  
}
```

```
module.exports = { addProduct, getProducts }; Output
```

:



RESULT: The RESTful API for Product Management was successfully implemented with GET, POST, PUT, and DELETE methods returning JSON responses.

TASK14: MICROSERVICES APP

AIM: To design a simple microservices-based system by splitting a monolithic application into independent services with loose coupling and single responsibility.

ALGORITHM: 1. Start the program

2. Identify modules in monolithic system
3. Split system into independent services
4. Create separate folders for each service
5. Implement Service 1 (Product Service)
6. Implement Service 2 (Order Service)
7. Use separate databases (db.js) for each service
8. Assign different ports for each service
9. Enable communication via REST APIs
10. Keep services loosely coupled
11. Deploy and run services independently 12. Access services and verify functionality

Program :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

  <title>Microservices App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>
```

```
<div class="container">

  <h2>Product Microservice</h2>

  <input type="text" id="name" placeholder="Product Name">

  <input type="number" id="price" placeholder="Price">

  <button onclick="addProduct()">Add</button>

  <hr>

  <button onclick="getProducts()">View Products</button>

  <div id="output"></div>

</div>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html> Style.css : body
```

```
{  background:
```

```
#1e1e2f;  color:
```

```
white;  font-family:
```

```
Arial;  text-align:
```

```
center;
```

```
}
```

```
.container {  width:
```

```
300px;  margin: 80px
```

```
auto;  background:
```

```
#2d2d44;  padding:
```

```
20px;  border-radius:
```

```
10px;
```

```
}
```

```
input {  width: 90%;
```

```
padding: 10px;
```

```
margin: 8px;
```

```
border: none;

border-radius: 5px;

}

button { padding: 8px
20px; margin: 10px;
background: #00bcd4;
color: white; border:
none; border-radius:
5px;
}

#output { margin-
top: 15px; text-
align: left;
}
```

Server.js :

```
const express = require("express"); const
dao = require("./db");

const app = express(); app.use(express.json());

// ADD PRODUCT

app.post("/products", (req, res) => {
  dao.addProduct(req.body, (err) => {
    if (err) return res.send("Error");
    res.send("Product Added (Service)");
  });
});
```

```
// GET PRODUCTS
```

```
app.get("/products", (req, res) => { dao.getProducts((err, data) => {    if (err) return  
res.send("Error");    res.json(data);  
    });  
});
```

```
app.listen(3001, () => { console.log("Product Service  
running on port 3001");  
});
```

Script.js :

```
const API = "http://localhost:3000/products"; function  
addProduct() { const name =  
document.getElementById("name").value; const price =  
document.getElementById("price").value;  
    fetch(API, {    method: "POST",    headers: {  
"Content-Type": "application/json" },    body:  
JSON.stringify({ name, price })  
    })  
    .then(res => res.text())  
    .then(msg => {  
alert(msg);  
getProducts();  
    });  
}  
function getProducts() {  
    fetch(API)  
    .then(res => res.json())  
    .then(data => {    document.getElementById("output").innerHTML =
```

```
        "<pre>" + JSON.stringify(data, null, 2) + "</pre>";
    });
}
```

DB.js :

```
const mysql = require("mysql2");

const db = mysql.createConnection({
  host: "localhost",  user: "root",
  password: "",  database:
    "micro_db"
});

db.connect(err => {  if (err) console.log("DB Error");
  else console.log("Product Service DB Connected");
});

function addProduct(data, cb) {
  db.query(
    "INSERT INTO products(name, price) VALUES (?, ?)",
    [data.name, data.price],
    cb
  );
}

function getProducts(cb) {
  db.query("SELECT * FROM products", cb);
}

module.exports = { addProduct, getProducts };
```

Output :

GET : <http://localhost:3001/products>

GET <http://localhost:3001/products>

200 OK

```
[
  null
]
```

POST: <http://localhost:3001/products>

GET <https://localhost:3001/products>

200 OK

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 50000
  },
  {
    "id": 2,
    "name": "Phone",
    "price": 200
  }
]
```

RESULTS: The application was successfully divided into independent microservices with separate responsibilities and communication via REST APIs.

TASK15: Service Registry App

Aim: To implement service registry and discovery using Node.js by dynamically registering and discovering microservices without hard-coded URLs.

ALGORITHM: 1. Start the program

2. Create a registry server

3. Store service name and URL in memory

4. Start Product Service

5. Register Product Service to registry

Program : Server.js

:

```
const express = require("express");
```

```
const app = express();
```

```
app.use(express.json()); // In-
```

```
memory registry let services = [];
```

```
/*
```

```
REGISTER SERVICE
```

Example:

```
{
```

```
  "name": "product-service",
```

```
  "url": "http://localhost",
```

```
  "port": 3001
```

```
}
```

```
*/ app.post("/register", (req, res) => {  const
```

```
service = req.body;  services.push(service);
```

```
console.log("Service Registered:", service);
```

```
res.send("Service Registered Successfully");
```

```
});
```

```

/*
GET ALL SERVICES

*/ app.get("/services", (req, res)
=> {   res.json(services);
});

/*
GET SINGLE SERVICE

*/ app.get("/services/:name", (req, res) => {   const
name = req.params.name;   const service =
services.find(s => s.name === name);

    if (service) {
res.json(service);

    } else {

        res.status(404).send("Service Not Found");

    }
});

/*
DELETE SERVICE

*/ app.delete("/services/:name", (req, res) => {

const name = req.params.name;   services =
services.filter(s => s.name !== name);

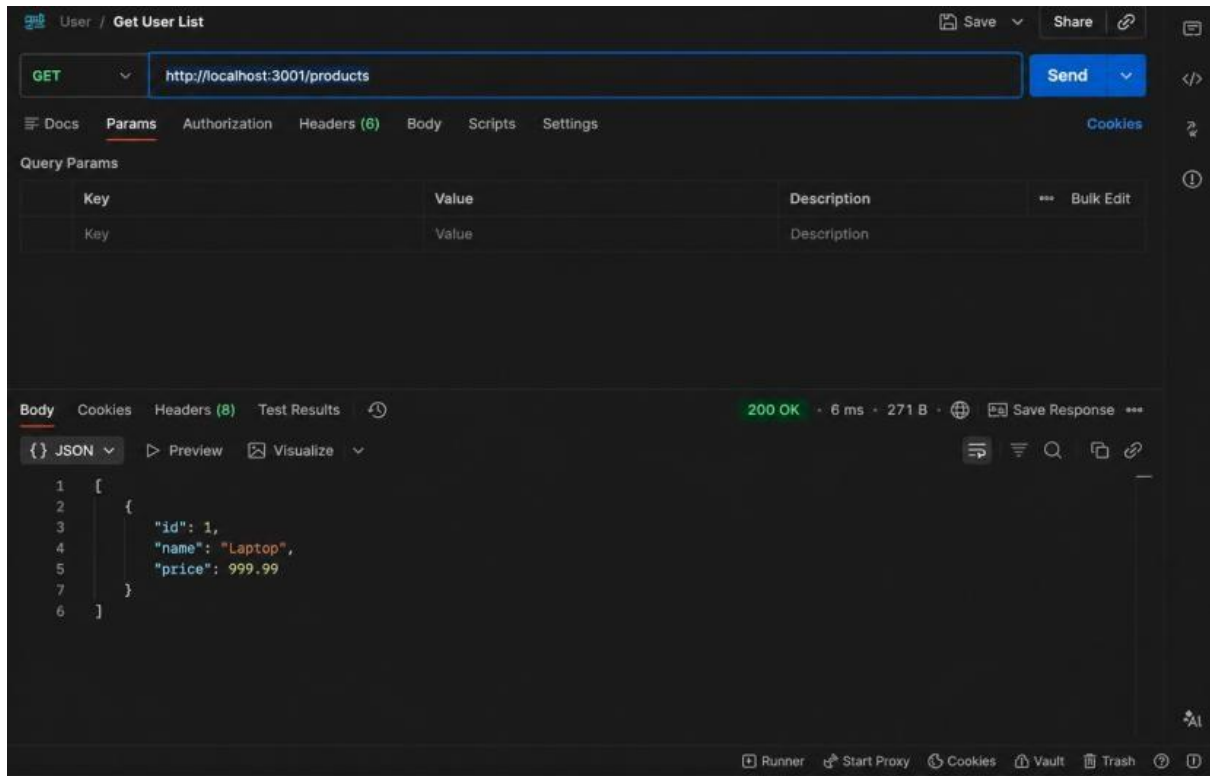
res.send("Service Removed");

});

app.listen(4000, () => {   console.log("Service Registry running on
http://localhost:4000"); });

```

Output :



RESULT: The microservices-based system was successfully implemented with service registry and discovery, enabling dynamic communication between independent services without hard coded URLs.

AIM: To configure an API Gateway that routes client requests to microservices and performs basic load balancing across multiple service instances.

ALGORITHM:

1. Start the program
2. Create API Gateway server
3. Define routes for microservices
4. Create multiple instances of a service
5. Store service URLs in an array
6. Implement round-robin load balancing
7. Receive client request at gateway
8. Select service instance dynamically
9. Forward request to selected instance
10. Get response from service
11. Send response back to client
12. Stop the program

Program : Server.js

```
:  
  
const express = require("express");  
  
const axios = require("axios"); const  
  
app = express();  
  
app.use(express.json());  
  
// Base service URL (Product Service) const  
  
PRODUCT_SERVICE = "http://localhost:3001/products";  
  
/*  
  
GET ALL PRODUCTS  
  
*/ app.get("/products", async (req, res)  
  
=> {  
  
  try {
```

```

        const response = await axios.get(PRODUCT_SERVICE);
res.json(response.data);

    } catch (err) {
res.status(500).send("Gateway Error");

    }
});

/*
ADD PRODUCT

*/ app.post("/products", async (req, res)
=> {

    try {

        const response = await axios.post(PRODUCT_SERVICE, req.body);
res.send(response.data);

    } catch (err) {
res.status(500).send("Gateway Error");

    }
});

/*
GET SINGLE PRODUCT

*/ app.get("/products/:id", async (req, res)
=> {

    try {

        const response = await axios.get(

            `${PRODUCT_SERVICE}/${req.params.id}`

        );

        res.json(response.data);

    } catch {
        res.status(404).send("Product
Not Found");

```

```

    }
  });

  /*
  DELETE PRODUCT

  */ app.delete("/products/:id", async (req, res)
=> {
    try {
      await axios.delete(`${PRODUCT_SERVICE}/${req.params.id}`);
      res.send("Product Deleted");
    } catch {
      res.status(404).send("Delete Failed");
    }
  });

  app.listen(3000, () => { console.log("API Gateway running at
  http://localhost:3000");
  });

```

Output :

```

{
  "status": "success",
  "users": [
    "Alice", "Bob", "Charlie"
  ]
}

```

```

{
  "id": 101,
  "product": "Wireless Headphones",
  "price": 59.99,
  "in_stock": true
}

```

RESULT: The API Gateway was successfully implemented to route requests and distribute load across multiple service instances using round-robin load balancing.

TASK17: Inter Service App

AIM:

To enable inter-service communication where one microservice consumes another service's REST API and handles responses and failures gracefully.

ALGORITHM:

1. Start the program
2. Create Product Service (provider)
3. Create Order Service (consumer)
4. Implement REST API in Product Service
5. Store Service product data in memory
6. Implement Order Service API
7. Use axios/fetch to call Product 8. Receive response from Product Service

FOLDER STRUCTURE:

inter-service-app/

|

|— product-service/

| └─ server.js

| └─ order-service/

| └─ server.js

Program :

product-service:server.js const

express = require("express"); const

app = express();

app.use(express.json()); // Sample

Product Data let products = [

{ id: 1, name: "Laptop", price: 50000 },

```

    { id: 2, name: "Phone", price: 200 }
  ];

  // GET ALL PRODUCTS app.get("/products",
  (req, res) => {    res.json(products);
  });

  // GET SINGLE PRODUCT app.get("/products/:id", (req, res)
  => {    const product = products.find(p => p.id ==
  req.params.id);    if (product) {      res.json(product);
    } else {
      res.status(404).send("Product Not Found");
    }
  });

  app.listen(3001, () => {    console.log("Product Service running at
  http://localhost:3001");
  });

order-service:server.js const express =
  require("express"); const axios =
  require("axios"); const app = express();
  app.use(express.json()); // CREATE
  ORDER (Inter-service call)
  app.post("/order", async (req, res) => {
  const productId = req.body.productId;
    try {
      // CALL PRODUCT SERVICE
      const response = await axios.get(
      `http://localhost:3001/products/${pr
      oductId}`
    );

```



```

        const product = response.data;    const
order = {    orderId:
Math.floor(Math.random() * 1000),
productName: product.name,    price:
product.price
    };
    res.json(order);  } catch (err) {    res.status(404).send("Product
not found from Product Service");
    }
});
app.listen(3002, () => {    console.log("Order Service running at
http://localhost:3002");
});

```

Output :

```

{
  "message": "Order created successfully",
  "order": {
    "id": 123456,
    "products": [
      { "id": 1, "name": "Laptop" },
      { "id": 2, "name": "Phone" }
    ]
  }
}

```

```

1  [
2    {
3      "id": 1,
4      "name": "Laptop"
5    },
6    {
7      "id": 2,
8      "name": "Phone"
9    }
10 ]

```

RESULT: Inter-service communication was successfully implemented using REST APIs, with proper handling of responses and failures between microservices.

TASK18: Microservice Test:

Aim :To write unit test cases for microservices to validate service logic, REST APIs, and data handling using a testing framework.

ALGORITHM:

1. Start the program
2. Install testing framework (Jest, Supertest)
3. Configure test script in package.json
4. Create test folder/files
5. Import service modules
6. Write test cases for service logic
7. Write test cases for REST APIs
8. Mock external dependencies if needed

Program : app.js:

```
const express = require("express");

const app = express();

app.use(express.json()); // Sample

Data let products = [

  { id: 1, name: "Laptop", price: 50000 },

  { id: 2, name: "Phone", price: 200 }

];

// GET ALL app.get("/products",

(req, res) => {

  res.json(products);

});

// GET ONE app.get("/products/:id", (req, res) => {  const

product = products.find(p => p.id == req.params.id);  if

(product) {    res.json(product);

  } else {

    res.status(404).send("Product Not Found");

  }

});
```

```
// ADD PRODUCT

app.post("/products", (req, res) => {

  const newProduct = req.body;

  newProduct.id = products.length + 1;

  products.push(newProduct);

  res.status(201).json(newProduct);

});

module.exports = app; server.js:

const app = require("./app"); const PORT = 3001;

app.listen(PORT, () => {  console.log(`Server running on

http://localhost:${PORT}`);

});
```

Product.Test.js :

```
const request = require("supertest"); const

app = require("./app"); describe("Product

API Test", () => {

  // TEST GET ALL  test("GET /products should return all

products", async () => {    const res = await

request(app).get("/products");

expect(res.statusCode).toBe(200);

expect(res.body.length).toBeGreaterThan(0);

  });

  // TEST GET ONE  test("GET /products/1 should return

product", async () => {    const res = await

request(app).get("/products/1");

expect(res.statusCode).toBe(200);

expect(res.body.name).toBe("Laptop");

  });
```

```

// TEST ADD    test("POST /products should add
product", async () => {    const res = await request(app)

    .post("/products")

    .send({ name: "Tablet", price: 300 });

expect(res.statusCode).toBe(201);    expect(res.body.name).toBe("Tablet");

});

// TEST NOT FOUND    test("GET /products/999 should
return 404", async () => {    const res = await
request(app).get("/products/999");

expect(res.statusCode).toBe(404);

});

});

```

Output :

```

PASS Product.test.js
✓ GET /products should return all products
✓ GET /products/1 should return product
✓ POST /products should add product
✓ GET /products/999 should return 404

Test Suites: 1 passed
Tests: 4 passed

```

RESULT: Unit test cases were successfully implemented to validate microservice logic, REST APIs, and data handling, ensuring application reliability.

TASK19: Mini Git System

AIM: To design a simple Git-like web application that tracks file changes, stores versions, and displays commit history using HTML, CSS, JavaScript, and Node.js.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for entering file content
4. Capture user input using JavaScript

5. Send data to server using fetch API
6. Store versions in database (array)
7. Assign version number to each commit
8. Save commit message and timestamp
9. Create API to fetch commit history
10. Display commit history in UI

Program :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

  <title>Mini Git System</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

  <h2>Mini Git System</h2>

  <!-- Commit Input -->

  <input type="text" id="message" placeholder="Enter commit message">

  <button onclick="addCommit()">Commit</button>

  <hr>

  <button onclick="getCommits()">View History</button>

  <div id="output"></div>

</div>

<script src="script.js"></script>

</body>

</html> Style.css : body

{  background:
```

```
#0d1117; color:
white; font-family:
Arial; text-align:
center;
}
.container { width:
320px; margin: 80px
auto; background:
#161b22; padding:
20px; border-radius:
10px;
}
input { width: 90%;
padding: 10px;
margin: 10px;
border-radius: 5px;
border: none;
}
button { padding:
8px 20px;
background:
#238636; color:
white; border:
none; border-
radius: 5px;
}
button:hover {
background: #2ea043;
```

```
}  
  
#output { margin-  
top: 15px; text-  
align: left; font-size:  
13px;  
}
```

Script.js :

```
const API = "http://localhost:3000/commits";  
  
// ADD COMMIT function addCommit() { const message =  
document.getElementById("message").value;  
  
  fetch(API, { method: "POST", headers: {  
"Content-Type": "application/json" }, body:  
JSON.stringify({ message })  
  
  })  
  
  .then(res => res.text())  
  
  .then(msg => {  
alert(msg);  
getCommits();  
  
  });  
}  
  
// GET COMMITS function  
getCommits() {  
fetch(API)  
  
  .then(res => res.json()) .then(data => { let output =  
"<pre>" + JSON.stringify(data, null, 2) + "</pre>";  
document.getElementById("output").innerHTML = output;  
  
  });  
}
```

DB.js :

```
let commits = []; // ADD COMMIT

function addCommit(data, cb) {

  const commit = {    id:

    commits.length + 1,    message:

    data.message,    time: new

    Date().toLocaleString()

    }

    commits.push(commit);

  cb(null, commit);

}

// GET COMMITS function

getCommits(cb) {

  cb(null, commits);

}

module.exports = { addCommit, getCommits };
```

Output :

Mini Git System

```
1  function getAll() {  
2      return commits;  
3  }  
4  
5  module.exports = { save, getAll };  
6  
7  
8
```

db.js

Commit

Commit History

- v1: Initial commit
- v2: db.js

RESULT: A mini Git-like system was successfully developed to track versions, store commits, and display history through a web interface.

TASK20: Mini Git Branch App

Aim: To develop a Git-like application that supports branching, merging, rebasing, and conflict resolution using a web interface.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Store branches and commits in memory
4. Create default branch (main)
5. Add feature to create new branch
6. Commit changes to selected branch
7. Implement merge logic
8. Detect conflicts (same file modified)

Program :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

  <title>Mini Git Branch</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="box">

  <h3>Mini Git Branch</h3>

  <input id="branch" placeholder="Branch name">

  <input id="msg" placeholder="Commit message">

  <button onclick="add()">Commit</button>

  <button onclick="view()">History</button>
```

```
<div id="out"></div>

</div>

<script src="script.js"></script>

</body>

</html> Style.css:

body {

background:#1e1e2f;

color:white; text-

align:center; font-

family:Arial;

} .box { margin:80px

auto; width:250px;

background:#2d2d44;

padding:15px;

border-radius:10px;

}

input {

width:90%;

margin:5px;

padding:8px;

}

button { margin:5px;

padding:6px 15px;

background:#00bcd4;

color:white;

border:none;

}
```

```

SCRIPT.JS: const API = "http://localhost:3000";

function add() { fetch(API + "/commit", {
  method:"POST",  headers:{ "Content-
Type":"application/json" },  body:
JSON.stringify({    branch: branch.value,
msg: msg.value

  })

}).then(()=>view());
}

function view() {
  fetch(API + "/commits")
    .then(r=>r.json()) .then(d=>{  out.innerHTML =
"<pre>"+JSON.stringify(d,null,2)+"</pre>";

  });
}

```

SERVER.JS:

```

const express = require("express"); const
cors = require("cors"); const db =
require("./db"); const app = express();
app.use(cors()); app.use(express.json());
app.post("/commit", (req,res)=>{
  db.add(req.body, ()=>res.send("Added"));
});
app.get("/commits", (req,res)=>{
  db.get(_data=>res.json(data));
});
app.listen(3000, ()=>console.log("Server running"));

```

DB.JS:

```

let commits = [];

exports.add = (data, cb) => {

  commits.push({  id:

    commits.length+1,

    branch: data.branch,

    msg: data.msg

  });

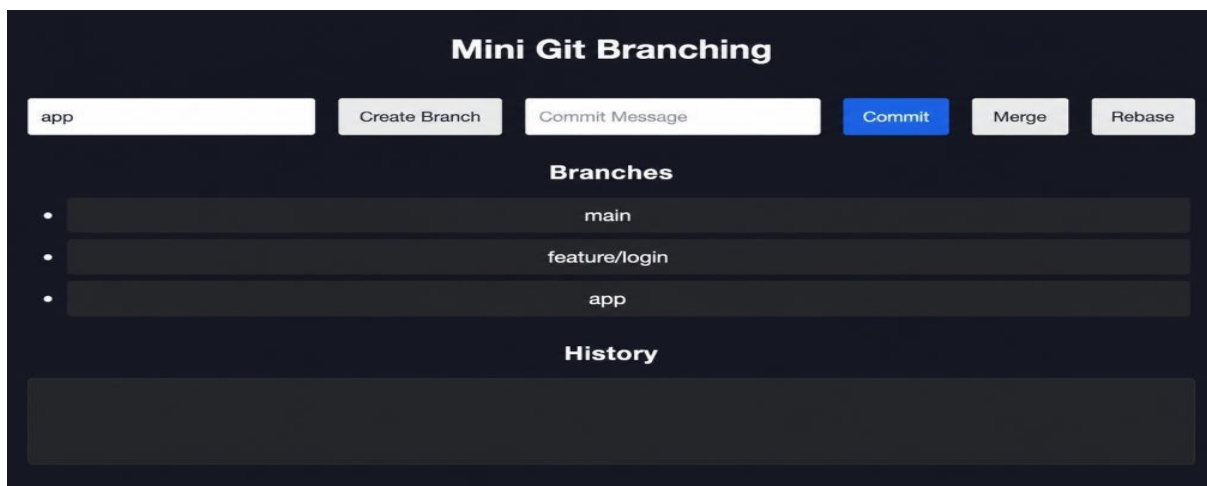
  cb(null);

};

exports.get = (cb) => cb(null, commits);

```

Output :



RESULT: A Git-like branching application was successfully developed to simulate branching, merging, rebasing, and conflict resolution through a web interface.

TASK21: Mini Cid App

AIM:

To develop a CI/CD pipeline simulation application that automates build, test, and deployment steps through a web interface.

ALGORITHM:

1. Start the program
2. Create project files (ML, CSS, JS, server.js, db.js)
3. Design UI for pipeline trigger
4. Capture user action (Run Pipeline)
5. Send request to backend
6. Simulate code checkout step
7. Simulate build process
8. Simulate test execution
9. Simulate deployment step
10. Store pipeline logs
11. Display pipeline status in UI
12. Stop the program

Program :

Index.html:

```
<!DOCTYPE html>

<html>

<head>

  <title>Mini CI/CD App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="box">

  <h3>Mini CI/CD Pipeline</h3>

  <button onclick="run('Build')">Build</button>

  <button onclick="run('Test')">Test</button>

  <button onclick="run('Deploy')">Deploy</button>

  <div id="out"></div>

</div>

<script src="script.js"></script>
```

```
</body>
```

```
</html> Style.css :
```

```
body {
```

```
background:#1e1e2f;
```

```
color:white; text-
```

```
align:center; font-
```

```
family:Arial;
```

```
} .box { margin:100px
```

```
auto; width:260px;
```

```
background:#2d2d44;
```

```
padding:20px;
```

```
border-radius:10px;
```

```
}
```

```
button { margin:5px;
```

```
padding:8px 15px;
```

```
background:#00bcd4;
```

```
color:white;
```

```
border:none; border-
```

```
radius:5px;
```

```
}
```

```
#out { margin-
```

```
top:15px; font-
```

```
size:14px;
```

```
}
```

```
Script.js :
```

```
function run(step) { const out =
```

```
document.getElementById("out");
```

```
out.innerHTML = step + " started...";
```

```
setTimeout(() => { out.innerHTML = step + "  
successful ";  
    }, 1500);  
}
```

Output :



RESULT: A CI/CD pipeline simulation application was successfully developed to automate and visualize build, test, and deployment processes.

TASK22: Cloud App

AIM: To develop a cloud comparison application that displays and compares services and pricing models of AWS, GCP, and Azure.

ALGORITHM:

1. Start the program

2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design user interface with dropdown for cloud providers
4. Initialize cloud service data (AWS, GCP, Azure) in database module
5. Capture selected provider using JavaScript
6. Send request to server to fetch service details
7. Display compute, storage, database, and networking services
8. Accept user input for compute hours and storage usage
9. Send input data to server for cost calculation 10. Calculate cost based on pay-as-you-use pricing model

Program :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

  <title>Cloud App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="box">

  <h3>Cloud Storage App</h3>

  <input id="name" placeholder="Enter Name">

  <input id="data" placeholder="Enter Data">

  <button onclick="save()">Save</button>

  <button onclick="load()">Load</button>

  <div id="out"></div>

</div> <script> const API =

"http://localhost:3000"; function save() {

fetch(API + "/save", {  method:"POST",
```

```
headers:{ "Content-Type":"application/json" },
body: JSON.stringify({    name: name.value,
data: data.value
    })
}).then(()=>load());
}
function load() {
fetch(API + "/data")
    .then(r=>r.json()) .then(d=>{    out.innerHTML =
"<pre>" +JSON.stringify(d,null,2)+"</pre>";
    });
}
</script>
</body>
</html> Style.css :
body {
background:#1e1e2f;
color:white; text-
align:center; font-
family:Arial;
} .box { margin:80px
auto; width:260px;
background:#2d2d44;
padding:20px;
border-radius:10px;
}
input {
width:90%;
```

```
margin:5px;

padding:8px;

}

button { margin:5px;

padding:6px 15px;

background:#00bcd4;

color:white;

border:none;

}

#out { margin-

top:15px; font-

size:13px;

}
```

Server.js :

```
const express = require("express");

const cors = require("cors"); const

db = require("./db");

const app = express(); app.use(cors());

app.use(express.json()); // SAVE DATA

app.post("/save", (req, res) => { db.save(req.body, ()

=> res.send("Saved to Cloud"));

});

// GET DATA app.get("/data", (req,

res) => { db.get(('_', data) =>

res.json(data));

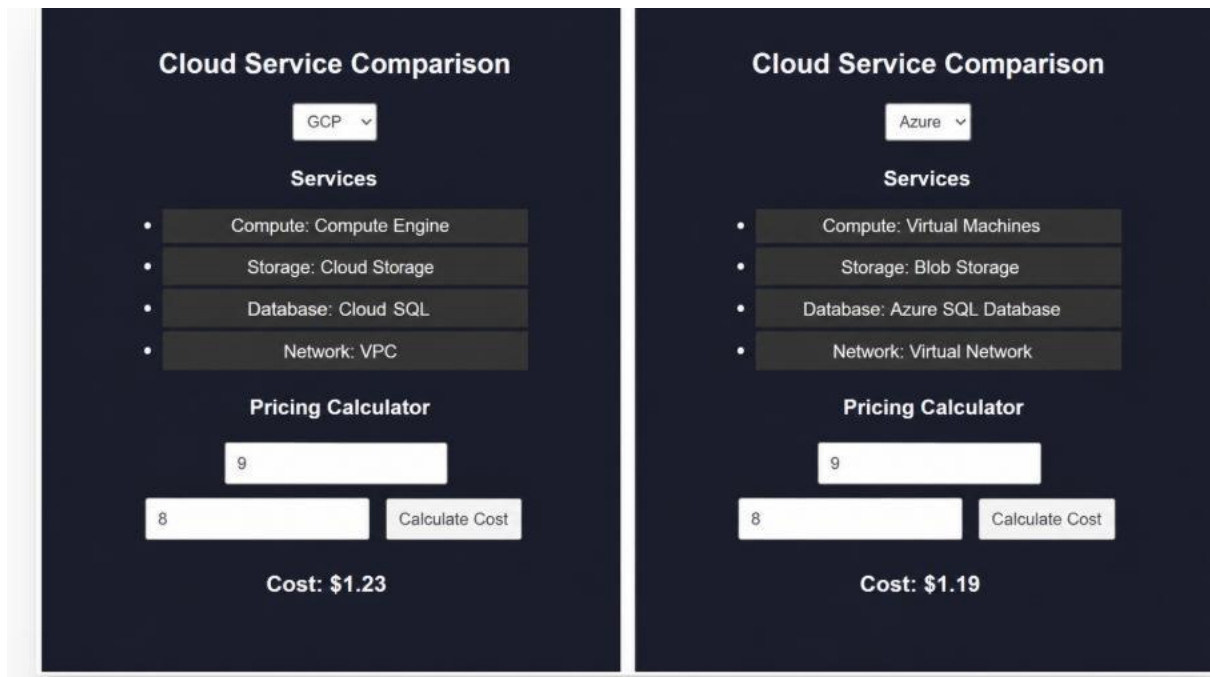
});

app.listen(3000, () => console.log("Cloud App running"));
```

```
DB.js : let storage = [];  
  
exports.save = (data, cb) => {  
  
  storage.push({ id:  
  
    storage.length + 1, name:  
  
    data.name, data:  
  
    data.data  
  
    });  
  
  cb(null);  
  
};  
  
exports.get = (cb) => {  
  
  cb(null, storage);  
  
};
```

Output :

The screenshot shows a web application titled "Cloud Service Comparison". At the top, there is a dropdown menu set to "AWS". Below this, a section titled "Services" contains a list of four items: "Compute: EC2", "Storage: S3", "Database: RDS", and "Network: VPC". Each item is preceded by a bullet point and is contained within a dark grey rectangular box. Below the services list is a section titled "Pricing Calculator". This section features two input fields: the first contains the number "9" and the second contains the number "8". To the right of the second input field is a button labeled "Calculate Cost". Below the input fields, the text "Cost: \$1.06" is displayed.



RESULTS: A cloud comparison application was successfully developed to display services and simulate pay-as-you-use pricing across AWS, GCP, and Azure.

TASK23: AI PROMPT APP

AIM: To develop an application that demonstrates prompt engineering by generating and evaluating code or cloud configurations based on user input.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI with input field for user prompt
4. Capture user prompt using JavaScript
5. Send prompt to server using fetch API

6. Receive prompt data in backend (server.js)
7. Process prompt using predefined logic (db.js)
8. Generate output (code / cloud config / response)
9. Evaluate output quality and assign score 10. Send generated output and score to frontend

Program :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

  <title>AI Prompt App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="box">

  <h3>AI Prompt App</h3>

  <input id="prompt" placeholder="Enter your prompt">

  <button onclick="send()">Ask AI</button>

  <div id="out"></div>

</div>

<script src="script.js"></script>

</body>

</html> Style.css :
```

```
body {

background:#1e1e2f;

color:white; text-

align:center; font-

family:Arial;
```

```
}.box { margin:100px  
auto; width:280px;  
background:#2d2d44;  
padding:20px;  
border-radius:10px;  
}
```

```
input {  
width:90%;  
padding:8px;  
margin:10px;  
}
```

```
button { padding:8px  
15px;  
background:#00bcd4;  
color:white;  
border:none;  
}
```

```
#out {  
margin-top:15px; font-  
size:14px;  
}
```

Script.js:

```
const API = "http://localhost:3000/ask"; function  
send() { fetch(API, { method:"POST",  
headers:{ "Content-Type":"application/json" },  
body: JSON.stringify({ prompt: prompt.value })  
})
```

```

    .then(r=>r.json()) .then(d=>{
out.innerHTML = "<b>AI:</b> " + d.reply;

    });
}

```

DB.js:

```

let history = []; exports.save =
(data, reply, cb) => {
  history.push({ id:
    history.length+1,  prompt:
    data.prompt,  reply: reply
  });
  cb(null);
};

exports.get = (cb) => cb(null, history); Server.js:

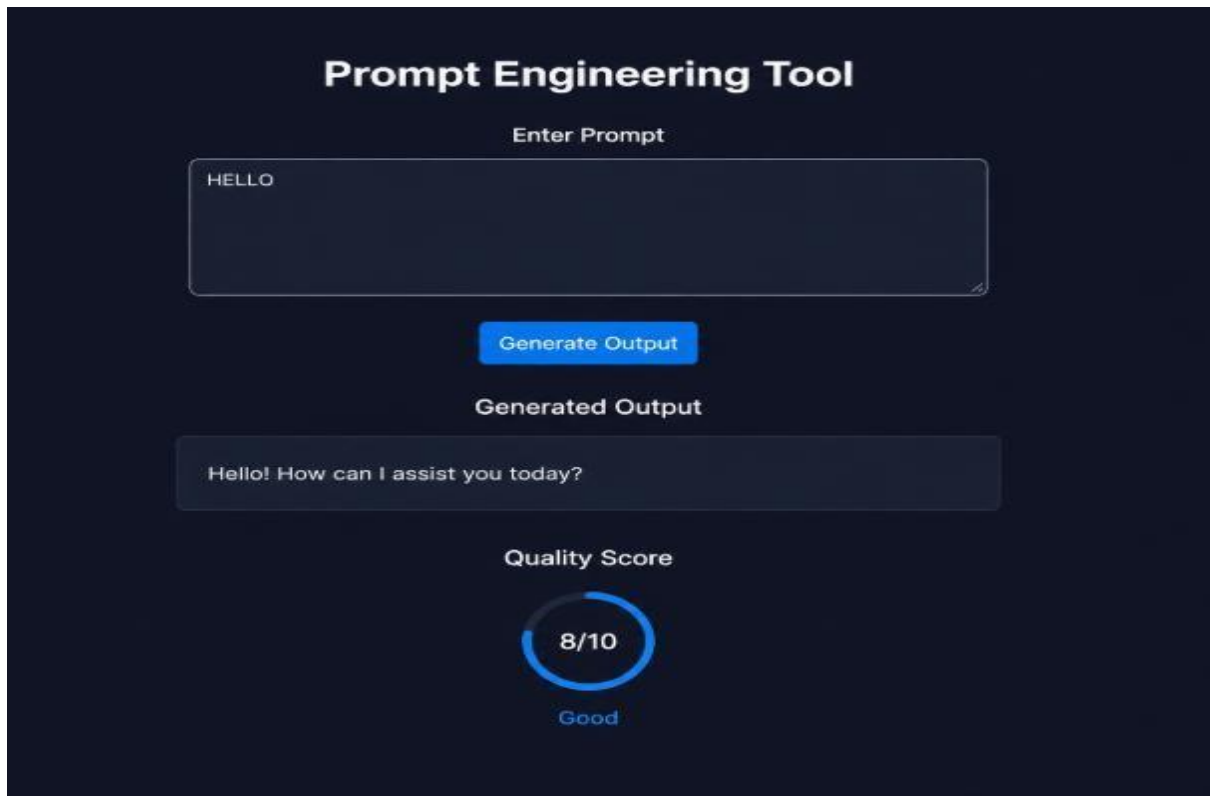
const express = require("express");
const cors = require("cors"); const
db = require("./db"); const app =
express(); app.use(cors());
app.use(express.json()); // FAKE AI
RESPONSE function
generateReply(prompt) { return "AI
Response to: " + prompt;
}

// ASK AI app.post("/ask", (req,res)=>{ const
reply = generateReply(req.body.prompt);
db.save(req.body, reply, ()=> {  res.json({ reply
});
});
});

```



```
});  
  
// GET HISTORY app.get("/history",  
  
(req,res)=>{  
  
db.get(('_data')=>res.json(data));  
  
});  
  
app.listen(3000, ()=>console.log("AI App running")); Output :
```



The screenshot displays a web application titled "Prompt Engineering Tool" on a dark blue background. It features a text input field labeled "Enter Prompt" containing the word "HELLO". Below this is a blue button labeled "Generate Output". Underneath the button is a section labeled "Generated Output" which contains a text box with the response "Hello! How can I assist you today?". At the bottom, there is a "Quality Score" section showing a circular progress indicator with the score "8/10" and the word "Good" below it.

RESULT: A prompt engineering application was successfully developed to generate and evaluate AI-based outputs for code and cloud configurations.

TASK24: Vibe Cloud App

AIM: To develop an application that uses iterative prompt engineering to generate and evaluate REST APIs, cloud deployment steps, and security configurations.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI to accept user prompts
4. Capture prompt input using JavaScript
5. Send prompt to backend using fetch API
6. Receive prompt in server and forward to processing module
7. Store prompt in history for version tracking
8. Analyze prompt type (REST API / Cloud / Security)
9. Generate corresponding output based on prompt
10. Evaluate output quality and assign score
11. Return output and score to frontend and display results
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Vibe Cloud App</title>
```

```
<link rel="stylesheet" href="style.css">
```

```
</head>

<body>

<div class="card">

  <h2> Vibe Cloud</h2>

  <input id="title" placeholder="Title">

  <input id="content" placeholder="Content">

  <button onclick="save()">Save</button>

  <button onclick="load()">Load</button>

  <div id="out"></div>

</div>

<script src="db.js"></script>

<script src="script.js"></script>

</body>

</html> Style.css : body { background: linear-
gradient(135deg, #1e1e2f, #3a3a6a); font-family: Arial;
text-align: center; color: white;
}

.card { width: 280px; margin:
100px auto; padding: 20px;
background: rgba(255,255,255,0.1);
border-radius: 15px; backdrop-
filter: blur(10px);
}

input { width: 90%;
padding: 8px;
margin: 8px;
border-radius: 5px;
border: none;
```

```
}  
  
button { margin: 5px;  
padding: 8px 15px;  
background: #00bcd4;  
border: none; color:  
white; border-radius:  
5px;  
}
```

```
#out { margin-  
top: 15px; font-  
size: 13px;  
}
```

Script.js : function save()

```
{ const data = { title:  
title.value, content:  
content.value  
};  
addData(data);  
load();  
}  
  
function load() { getData((d) => { out.innerHTML = "<pre>" +  
JSON.stringify(d, null, 2) + "</pre>";  
});  
}
```

DB.js :

```
let cloud = []; function  
addData(data) {  
cloud.push({ id:
```

```
cloud.length + 1, title:
```

```
data.title, content:
```

```
data.content
```

```
});
```

```
}
```

```
function getData(cb) {
```

```
cb(cloud);
```

```
}
```

Output :



RESULT: A Vibe Coding application was successfully developed to generate and refine cloud based features using iterative prompt engineering.